Proceedings of the 2026 International Conference on Emerging Trends in Mobile Computing and Sustainable Informatics (ICEMCSI)
IEEE Xplore Part Number: CFP263AB-ART; ISBN: 979-8-3315-5336-4

LLM Based Cognitive AIOps Autonomous Cloud Incident Management

Kavitha K S
Department of Computer Science and Engineering
Dayananda Sagar College of Engineering
Bengaluru, India
dr.kavitha-cse@dayanandasagar.edu.in

Nagraj M Lutimath
Department of Computer Science and Engineering
Dayananda Sagar Academy of Technology and Management
Bengaluru, India
nagarajml-cse@dsatm.edu.in

Akash Singh
Department of Computer Science and Engineering
Dayananda Sagar College of Engineering
Bengaluru, India
akashsingh2710670@gmail.com

Ayush Gupta
Department of Computer Science and Engineering
Dayananda Sagar College of Engineering
Bengaluru, India
*a*yushgupta1312005@gmail.com

Adi Hema Ajay Charan
Department of Computer Science and Engineering
Dayananda Sagar College of Engineering
Bengaluru, India
adiajay12367@gmail.com

Ananya Gupta
Department of Computer Science and Engineering
Dayananda Sagar College of Engineering
Bengaluru, India
ananyagupta8303@gmail.com

***Abstract*—DreamOps is an AI-powered platform for managing incidents and infrastructure automatically. It uses Anthropic Claude, a large language model integrated via the Model Context Protocol, to handle cloud-native issues. The system works with Kubernetes, PagerDuty, Grafana, GitHub, Slack, and Notion, among others. This lets DreamOps understand different infrastructure data much better. The platform handles problems in three steps: it tries to fix them on its own first, then moves to planning with human help if needed, and finally, for risky decisions, it gets final approval from humans too. It makes choices based on how confident it is about each action, keeping things safe and reliable. What sets DreamOps apart is its use of LLM- driven thinking for handling new incidents. This dynamic approach improves results dramatically. Tests showed that incidents got fixed 80% faster, and there was a 90% drop in late-night calls for engineers. Plus, the fixes became more consistent. So, DreamOps could really change DevOps by making processes smarter and partly automatic, shifting focus from reacting to being proactive and efficient.**

Keywords— ***Incident response automation, large language models, DevOps, Model Context Protocol, Kubernetes, AI agents, infrastructure management, on- call engineering, MTTR reduction, cloud-native systems.***

I. Introduction

The proliferation of microservices architectures, containerization, and distributed cloud systems has fundamentally transformed the operational landscape of modern software engineering. While these paradigms deliver unprecedented scalability and deployment velocity, they introduce a commensurate increase in operational complexity and failure surface area. Production systems may fail across dozens of interacting services, generating a cascade of correlated alerts that overwhelm conventional monitoring pipelines and human operators alike [1].

On-call engineering has consequently emerged as a critical—and deeply taxing—operational role. Engineers designated for on-call duty are expected to respond to production incidents at any hour, diagnosing root causes under severe time pressure and cognitive load. Surveys of DevOps practitioners consistently report that on-call stress contributes to engineer burnout, reduced job satisfaction, and elevated attrition rates [2]. The cost of this human toll is compounded by the economic impact of production downtime, which averages USD 5,600 per minute across enterprise systems [3].

Current incident management tooling—comprising alerting platforms such as PagerDuty and OpsGenie, observability stacks such as Grafana and Datadog, and runbook-based response procedures—provides substantial data collection and notification capabilities but leaves the analytical and decision- making burden squarely on human operators. Engineers must manually correlate log streams, query metrics dashboards, parse documentation, and formulate remediation steps, all while under pressure to minimize downtime.

Recent advances in large language models (LLMs) present a compelling opportunity to augment or automate these cognitive tasks. Models such as Anthropic Claude demonstrate sophisticated reasoning capabilities over unstructured text, structured data, and multi-modal inputs, enabling them to synthesize information from disparate sources and generate contextually appropriate responses [4]. The Model Context Protocol (MCP) further extends these capabilities by providing a standardized interface for LLMs to interact with external tools, APIs, and knowledge bases in a composable and auditable manner.

On-call engineering has consequently emerged as a critical— and deeply taxing—operational role. Engineers designated for on-call duty are expected to respond to production incidents at any hour, diagnosing root causes under severe time pressure and cognitive load. Surveys of DevOps practitioners consistently report that on-call stress contributes to engineer burnout, reduced job satisfaction, and elevated attrition rates [2]. The cost of this human toll is compounded by the economic impact of production downtime, which averages USD 5,600 per minute across enterprise systems [3].

Current incident management tooling—comprising alerting platforms such as PagerDuty and OpsGenie, observability stacks such as Grafana and Datadog, and runbook-based response procedures—provides substantial

2026 International Conference on Emerging Trends in Mobile Computing and Sustainable Informatics (ICEMCSI) | 979-8-3315-5336-4/26/$31.00 ©2026 IEEE | DOI: 10.1109/ICEMCSI67638.2026.11603148

979-8-3315-5336-4/26/$31.00 ©2026 IEEE

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:38:32 UTC from IEEE Xplore. Restrictions apply.

data collection and notification capabilities but leaves the analytical and decision- making burden squarely on human operators. Engineers must manually correlate log streams, query metrics dashboards, parse documentation, and formulate remediation steps, all while under pressure to minimize the downtime.

Recent advances in large language models (LLMs) present a compelling opportunity to augment or automate these cognitive tasks. Models such as Anthropic Claude demonstrate sophisticated reasoning capabilities over unstructured text, structured data, and multi-modal inputs, enabling them to synthesize information from disparate sources and generate contextually appropriate responses [4]. The Model Context Protocol (MCP) further extends these capabilities by providing a standardized interface for LLMs to interact with external tools, APIs, and knowledge bases in a composable and auditable manner[5].

This paper introduces DreamOps, an autonomous incident response platform that integrates Claude-based LLM reasoning with MCP-mediated tool use to create a fully automated DevOps co-pilot. DreamOps addresses the following core research questions:

RQ1: Can LLM-based agents achieve reliable automated incident resolution for common infrastructure failure classes without human intervention?

RQ2: What confidence and risk thresholds are appropriate for autonomous command execution in production environments?

RQ3: How can multi-source context aggregation from heterogeneous DevOps tools improve root cause analysis accuracy?

RQ4: What system architecture best balances autonomy, safety, and human oversight in AI- driven incident response?

Our contributions are threefold. First, we present a novel three-tier response architecture that allows organizations to calibrate the degree of AI autonomy based on operational risk tolerance. Second, we introduce a confidence-risk scoring framework that governs autonomous command execution, empirically validated across a corpus of simulated and production incidents. Third, we describe the complete system design, implementation details, and evaluation methodology for DreamOps, which is available as an open-source platform.

The remainder of this paper is organized as follows. Section II reviews related work in automated incident response and LLM-based agent systems. Section III describes the DreamOps system architecture and key components. Section IV details the MCP integration layer and supported tool ecosystem. Section V presents the confidence-risk scoring model. Section VI describes the experimental setup and evaluation methodology. Section VII reports empirical results and analysis. Section VIII discusses limitations and future directions. Section IX concludes.

II. Related Work

A. *Automated Incident Management*

The automation of incident detection and response has been an active research area for over two decades Early work by Feather et al. [6] explored rule-based expert systems for fault diagnosis in distributed systems, demonstrating the feasibility of automated root cause analysis for well-defined failure taxonomies. Subsequent work by Cohen et al. [7] introduced machine learning approaches to anomaly detection, leveraging statistical models to identify deviations from baseline system behaviour.

More recently, AIOps platforms have emerged as a category of commercial and research tools that apply machine learning to operational data for alert correlation, noise reduction, and predictive fault detection [8]. Tools such as Moogsoft and BigPanda implement event correlation algorithms that reduce alert volumes by grouping related signals into incident clusters. However, these systems primarily address the detection and triage phases of incident management; the remediation phase continues to rely on human judgment and execution.

IBM's Watson AIOps [9] represents a more ambitious approach, integrating natural language processing to parse runbook documentation and suggest remediation steps. Nevertheless, Watson AIOps operates in an advisory capacity, presenting recommendations to human operators rather than autonomously executing fixes. DreamOps extends this paradigm by providing a fully autonomous execution capability with configurable risk controls.

B. *LLM-Based Agent Systems*

The application of LLMs to agentic tasks—wherein models must plan, execute actions, and observe results over multi- step trajectories—has attracted substantial recent attention. ReAct [10] introduced a prompting framework that interleaves reasoning traces with action executions, enabling LLMs to use tools and observe feedback in a structured loop. Subsequent work on Toolformer [11] demonstrated that LLMs could learn to call external APIs through self-supervised training.

The emergence of the Model Context Protocol (MCP) [5] represents a significant architectural advance, providing a standardized, composable interface for LLMs to integrate with arbitrary external tools. MCP defines a client-server protocol in which tool capabilities are described via a structured schema, enabling LLMs to discover available tools, formulate appropriate calls, and process structured results. DreamOps leverages MCP to integrate with ten DevOps tools through a unified interface, avoiding the brittle tool-specific integration code that characterizes earlier approaches.

Concurrent work on SWE-agent [12] and Devin [13] demonstrates the application of LLM agents to software engineering tasks, including code generation, debugging, and repository navigation. While these systems operate on software artifacts, DreamOps applies analogous agent architectures to infrastructure management, where actions have immediate real-world consequences requiring careful safety controls.

C. *Self-Healing Infrastructure*

The concept of self-healing infrastructure—systems that automatically detect and recover from failures—has been pursued through various mechanisms. Kubernetes itself incorporates primitive self-healing capabilities, including pod restart policies, liveness probes, and horizontal pod autoscaling [14]. However, these mechanisms operate on predefined rules and cannot adapt to novel failure modes or complex multi-service interactions.

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:38:32 UTC from IEEE Xplore. Restrictions apply.

Chaos engineering platforms such as Chaos Monkey [15] have advanced our understanding of system resilience by systematically injecting failures and observing system responses. DreamOps complements this approach by providing an intelligent response layer that can leverage chaos engineering insights, stored in integrated knowledge bases, to inform remediation strategies.

III. System Architecture

A. *Architectural Overview*

DreamOps employs a layered microservices architecture comprising five principal components: the Alert Ingestion Layer, the AI Agent Core, the MCP Integration Layer, the Risk and Confidence Engine, and the User Interface. Figure 1 depicts the high-level system architecture and data flow.

The system is designed for cloud-native deployment on AWS ECS and EKS, with full Docker containerization and Terraform-based infrastructure provisioning. All inter-service communication occurs over authenticated HTTPS channels with end-to-end TLS encryption. Secrets management follows AWS Secrets Manager best practices with complete environment separation between development, staging, and production deployments.

B. *Alert Ingestion Layer*

The Alert Ingestion Layer serves as the primary entry point for incident signals. DreamOps exposes a webhook endpoint that receives PagerDuty incident payloads conforming to the PagerDuty Events API v2 specification.

Upon receipt, the ingestion service performs payload validation, deduplication using incident UUID comparison against a Redis cache, and priority classification based on PagerDuty severity levels (P1–P4).

The ingestion layer implements a tiered alert throttling mechanism to prevent alert storms from overwhelming the AI agent. During periods of high alert volume, alerts are queued in an Amazon SQS queue with visibility timeout configured to match expected agent processing latency. This design ensures that alert processing remains bounded and predictable even under degraded system conditions.

A configurable master toggle (AI_AGENT_ENABLED environment variable) enables operators to completely disable AI analysis at the infrastructure level, taking precedence over all application-level controls. This provides a critical safety valve for situations in which AI processing must be suspended immediately, such as during sensitive maintenance windows or in response to AI system anomalies.

The AI Agent Core constitutes the central reasoning component of DreamOps. It is implemented as a Python FastAPI service that orchestrates interactions between the Claude LLM and the MCP tool ecosystem. The agent operates on a ReAct-inspired [10] reasoning loop, iteratively formulating hypotheses about incident root causes, gathering corroborating evidence through tool calls, and updating its working hypothesis until sufficient confidence is achieved.

The agent receives an enriched incident context object comprising the PagerDuty alert payload, a sliding window of recent similar incidents from the incident history database, relevant runbook excerpts retrieved via semantic search from the Notion knowledge base, and current Kubernetes cluster state including pod status, resource utilization, and recent event logs. This multi- source context enables the agent to reason about incidents with a breadth of information equivalent to an experienced engineer conducting manual investigation.

Claude's extended context window (200,000 tokens in Claude 3 models) is particularly advantageous for this application, as it allows the agent to reason simultaneously over large log files, multi-service dependency graphs, and extensive runbook documentation without information truncation. The agent's system prompt encodes domain-specific knowledge about Kubernetes failure patterns, common remediation strategies, and risk classification guidelines developed through empirical analysis of production incident data.

C. *Response Mode Framework*

DreamOps implements three distinct response modes that organizations can configure based on their risk tolerance and operational maturity:

YOLO Mode (Autonomous Execution): The agent autonomously executes remediation commands without human approval for actions classified as low-risk with confidence scores exceeding the configured threshold (default: 80%). This mode is designed for organizations with mature infrastructure automation practices and well-understood failure taxonomies. All actions are logged with full audit trails.

IV. MCP Integration Layer

A. *Protocol Architecture*

The Model Context Protocol provides a standardized client- server architecture for LLM tool use. Each external system is wrapped in an MCP server that exposes a set of typed tools discoverable by the AI agent. The DreamOps MCP integration layer currently implements servers for six primary systems: Kubernetes cluster management, PagerDuty incident operations, Grafana metrics and alerting, GitHub repository and pull request access, Slack messaging, and Notion knowledge base retrieval.

MCP tool definitions encode parameter schemas, return type specifications, and natural language descriptions that guide the agent's tool selection and parameterization. The agent dynamically selects among available tools based on the current investigation context, forming multi-step tool chains that mirror the investigative workflows of experienced DevOps engineers.

B. *Kubernetes MCP Server*

The Kubernetes MCP server exposes tools for cluster state inspection and remediation execution. Read operations include pod listing with status filtering, log retrieval with configurable line limits and timestamp ranges, deployment and service specification retrieval, resource utilization queries via the Kubernetes Metrics API, and event log retrieval filtered by namespace, resource type, and time window.

Write operations—pod deletion, deployment scaling, configuration map updates, and resource limit adjustments—are subject to the Risk and Confidence Engine's approval requirements before execution. The Kubernetes MCP server implements a dry-run capability for all write operations,

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:38:32 UTC from IEEE Xplore. Restrictions apply.

allowing the agent to validate command correctness before execution without causing state changes.

C. *Grafana MCP Server*

The Grafana MCP server enables the agent to query time-series metrics and alerting state from Grafana Cloud and self-hosted Grafana instances. The server implements PromQL query execution, allowing the agent to formulate and execute metric queries dynamically based on the incident context. Automated query templates covering common diagnostic patterns—CPU saturation, memory pressure, request error rates, latency percentiles, and

saturation metrics following the RED method [17]—reduce query formulation time and improve diagnostic consistency.

Dashboard panel data retrieval allows the agent to extract structured metric summaries that are injected into the LLM context as tabular data, enabling quantitative reasoning about system state changes over time. This capability is particularly valuable for identifying gradual resource exhaustion patterns that precede catastrophic failures.

D. *Knowledge Base Integration*

The Notion MCP server provides access to organizational runbooks, architecture documentation, and post-mortem reports stored in Notion workspaces. DreamOps implements a semantic retrieval pipeline that embeds incident descriptions and tool outputs using sentence transformers, then performs approximate nearest-neighbor search against a pre-indexed corpus of documentation to surface relevant runbook sections.

This retrieval-augmented generation (RAG) approach allows the agent to leverage organizational knowledge accumulated over years of operational experience, avoiding the hallucination risks associated with purely parametric LLM reasoning about organization-specific infrastructure configurations and procedures. Post- mortem reports are particularly valuable as they encode hard-won diagnostic insights from past incidents.

CONFIDENCE-RISK SCORING MODEL

A. *Model Design*

A central challenge in autonomous incident response is determining when AI-generated remediation actions can be safely executed without human review. Executing an incorrect remediation action on a production system can exacerbate an incident, extend downtime, or cause data loss. Conversely, requiring human approval for all actions eliminates the latency benefits of automation.

DreamOps addresses this challenge through a two-dimensional confidence-risk scoring model. Confidence scores (range: 0.0–1.0) reflect the agent's self-assessed certainty in its root cause diagnosis and proposed remediation, derived from the consistency of supporting evidence across multiple data sources, the similarity of the current incident to resolved historical incidents, and the semantic similarity of retrieved runbook content to the proposed action.

Risk scores (levels: LOW, MEDIUM, HIGH, CRITICAL) classify the potential impact of a proposed action based on a predefined taxonomy of Kubernetes and infrastructure operations. Pod deletions in stateless deployments with replicas > 1 are classified LOW; deployment scaling operations are classified MEDIUM; persistent volume modifications and network policy changes are classified HIGH; node draining and cluster- level operations are classified CRITICAL.

The execution policy maps (confidence, risk) pairs to response mode requirements. In YOLO Mode, actions with confidence >= 0.80 and risk <= MEDIUM are executed autonomously. Actions with confidence >= 0.70 and risk = HIGH require Slack notification with a brief acknowledgment period (default: 2 minutes) before execution. Actions with CRITICAL risk are always escalated to human approval regardless of confidence score.

This policy was derived through empirical analysis of 340 simulated incidents across a representative Kubernetes workload, where the optimal thresholds were identified by maximizing the F1 score of the joint objective of automation rate (proportion of incidents resolved without human intervention) and resolution correctness (proportion of resolved incidents where the executed action was later confirmed correct by human review).

V. EXPERIMENTAL EVALUATION

A. *Experimental Setup*

We evaluate DreamOps across two experimental settings: a controlled laboratory environment with simulated infrastructure failures, and a production deployment at a mid-sized software organization with approximately 200 engineers. The laboratory environment consists of a 5-node Kubernetes cluster running 40 microservices spanning a representative e-commerce workload, instrumented with Prometheus and Grafana for metrics collection.

We constructed a corpus of 340 incident scenarios covering the 12 most common Kubernetes failure classes identified in our analysis of public post-mortem databases: CrashLoopBackOff, OOMKilled, ImagePullBackOff, pending pods due to resource insufficiency, service endpoint unavailability, persistent volume claim failures, config map and secret misconfiguration, horizontal pod autoscaler thrashing, ingress routing failures, certificate expiration, node NotReady, and etcd performance degradation.

For the production deployment, we collected incident data over a 6-month period from January to June 2025, comprising 847 PagerDuty incidents across the organization's AWS EKS infrastructure. DreamOps operated in Plan Mode for the first two months to build organizational confidence, transitioning to YOLO Mode for low-risk incidents in months three through six.

B. *Baseline Comparisons*

We compare DreamOps against three baselines: (1) Human- only incident response following the organization's existing on-call procedures; (2) Rule-based automation using PagerDuty Event Intelligence with predefined automation rules; and (3) LLM-advisory mode, in which Claude generates remediation recommendations posted to Slack but all execution is performed manually by engineers.

C. *Evaluation Metrics*

We evaluate system performance across four primary metrics: Mean Time to Resolution (MTTR), measured from

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:38:32 UTC from IEEE Xplore. Restrictions apply.

alert creation to incident resolution; Automation Rate, defined as the proportion of incidents resolved without human execution of remediation commands; Resolution Correctness, assessed by post-incident review comparing executed actions against expert-determined optimal remediation; and Engineer Interruption Rate, measured as after-hours escalations per on-call shift.

VI. Results And Analysis

A. *Incident Resolution Performance*

Table I presents the primary evaluation results across all experimental conditions. DreamOps in YOLO Mode achieves a mean MTTR of 4.3 minutes for common incident classes, compared to 47.2 minutes for human-only response—an 80.9% reduction. The improvement is particularly pronounced for the five highest-frequency incident classes (CrashLoopBackOff,OOMKilled, ImagePull BackOff, resource insufficiency, and service unavailability), which collectively constitute 68% of the incident corpus.

TABLE I. Incident Resolution Performance E Comparison

System	*MTT R (min)*	*Auto Rate (%)*	*Correctness (%)*	*Interruption s /shift*
Human Only	47.2	0	91.3	4.7
Rule-Based	38.4	22.1	83.7	3.2
LLM Advisory	22.8	0	94.6	2.9
DreamOps (Plan)	11.3	71.4	96.2	1.4
)reamOps (YOLO)	4.3	87.6	95.8	0.5

Resolution correctness for DreamOps in YOLO Mode (95.8%) is comparable to LLM Advisory mode (94.6%) and both exceed rule-based automation (83.7%), demonstrating that the confidence-risk scoring framework effectively prevents incorrect autonomous actions. The marginal correctness difference between YOLO Mode and Plan Mode (95.8% vs. 96.2%) suggests that human review adds modest but measurable benefit for edge cases.

B. *Production Deployment Results*

In the 6-month production deployment, DreamOps processed 847 PagerDuty incidents. Of these, 74.3% were autonomously resolved in YOLO Mode without engineer intervention, 18.7% were resolved through Plan Mode with engineer plan approval, and 7.0% required direct human diagnosis and remediation due to novel failure modes outside the agent's confident handling range. The 90% reduction in after-hours escalations (from 4.7 to 0.5 interruptions per shift) was statistically significant ($p < 0.001$, paired t-test), representing the most operationally impactful outcome of the deployment.

C. *Confidence-Risk Model Validation*

Analysis of the confidence-risk scoring model reveals that 94.1% of incidents classified as LOW risk with confidence >= 0.80 were resolved correctly in autonomous mode, validating the chosen execution thresholds. The false positive rate—actions executed autonomously that were later determined incorrect— was 4.2%, with all false positive cases resulting in auto-correcting Kubernetes pod restarts with no persistent negative impact. No MEDIUM or HIGH risk false positives resulted in data loss or extended downtime

D. *Multi-Source Context Analysis*

Ablation experiments assessed the contribution of individual context sources to resolution correctness. Removing Grafana metrics context reduced correctness by 8.3 percentage points for resource-related incidents. Removing runbook retrieval reduced correctness by 11.7 points for configuration-related incidents. Removing incident history reduced correctness by 6.2 points across all categories. These results confirm that multi-source context aggregation is essential for reliable automated diagnosis and that no single data source is sufficient.

VII. Discussion

A. *Implications for DevOps Practice*

DreamOps demonstrates that LLM-based agents can achieve autonomous incident resolution quality exceeding rule-based automation while approaching human expert performance for well-characterized failure classes. The system's 80.9% MTTR reduction directly translates to reduced customer impact: for an organization experiencing USD 5,600 per minute downtime costs, resolving a common incident in 4.3 minutes rather than 47.2 minutes saves approximately USD 240,000 per incident [3]. The 90% reduction in after-hours engineer escalations has profound implications for workforce wellbeing. On-call engineers at the production deployment site reported significantly reduced sleep disruption and on-call anxiety in post-deployment surveys, with 87% rating the system as net positive for their work-life balance. These qualitative outcomes complement the quantitative performance improvements and suggest that AI-assisted incident response may help address the systemic burnout challenges facing the DevOps profession.

B. *Safety and Trust Considerations*

The deployment of autonomous systems in production environments raises important questions about appropriate human oversight and accountability. Our three-tier response mode framework represents one answer to this challenge, allowing organizations to calibrate autonomy to their risk tolerance. We recommend that organizations new to AI-assisted incident response begin with Plan Mode or Approval Mode to develop trust in system behavior before enabling YOLO Mode.

The confidence-risk scoring model provides a principled mechanism for autonomous execution decisions, but it is not infallible. The 4.2% false positive rate, while low, indicates that autonomous execution will occasionally produce incorrect actions. Robust rollback capabilities, comprehensive audit logging, and conservative risk classification of ambiguous operations are essential safeguards for production deployments.

C. *Limitations*

Several limitations of the current DreamOps implementation warrant acknowledgment. First, the system's performance is bounded by its MCP tool coverage; incidents involving systems not yet integrated require human intervention. Second, the confidence scoring model was calibrated on a specific workload profile and may require recalibration for organizations with substantially different infrastructure patterns. Third, the system does not currently support multi-cluster federation, limiting applicability for

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:38:32 UTC from IEEE Xplore. Restrictions apply.

organizations operating across geographically distributed Kubernetes deployments.

Additionally, the current evaluation is limited to Kubernetes-based infrastructure. Cloud provider- specific incident types (AWS service disruptions, GCP networking issues) are not yet handled, representing a significant gap given the prevalence of managed cloud services in modern infrastructure stacks.

D. *Future Directions*

Several promising research directions emerge from this work. Predictive incident prevention—detecting precursor signals of impending failures before alerts are triggered—represents a natural extension of the current reactive architecture. Multi-cloud support encompassing AWS, GCP, and Azure service APIs would substantially expand the system's applicability. Fine- tuning Claude on domain-specific incident data may further improve diagnostic accuracy for specialized infrastructure configurations.

Federated learning approaches could enable the development of shared incident knowledge bases across organizations while preserving proprietary operational data, creating network effects that improve system performance proportionally to community participation.

VIII. CONCLUSION

This paper presented DreamOps, an autonomous AI-powered incident response platform that integrates Anthropic Claude with MCP-mediated tool use to automatically triage, diagnose, and remediate infrastructure incidents. Our evaluation demonstrates that DreamOps achieves an 80.9% reduction in mean time to resolution, an 87.6% autonomous resolution rate, and a 90% reduction in after-hours engineer escalations, while maintaining resolution correctness comparable to human expert performance. The confidence-risk scoring framework provides a principled mechanism for autonomous execution decisions, validated through empirical analysis across 1,187 incident scenarios. The three-tier response mode architecture enables organizations to deploy AI-assisted incident response with a level of autonomy appropriate to their operational maturity and risk tolerance. DreamOps demonstrates that LLM-based agent systems have matured to the point where they can provide substantial, reliable value in high-stakes production environments.

REFERENCES

[1] L. A. Barroso, U. Hölzle, and P. Ranganathan, The Datacenter as a Computer: Designing Warehouse-Scale Machines, 3rd ed. San Rafael, CA, USA: Morgan & Claypool, 2018.

[2] T. Limoncelli, S. Chalup, and C. Hogan, "The Practice of Cloud System Administration: Designing and Operating Large Distributed Systems," Addison-Wesley, 2014.

[3] Gartner. (2014). The cost of IT downtime. Research conducted by Gartner, Inc.

[4] Anthropic, "Claude's Model Card: Technical Report on Claude 3," Anthropic Technical Report, 2024.

[5] Anthropic, "Model Context Protocol Specification v1.0," Anthropic, Nov. 2024.

[6] M. Feather, S. Fickas, A. van Lamsweerde, and C. Ponsard, "Reconciling System Requirements and Runtime Behavior," Proc. IWSSD, pp. 50–61, 1998.

[7] P. Notaro, J. Cardoso, and M. Gerndt, "A survey of AIOps methods for failure management," ACM Trans. Intell. Syst. Technol., vol. 12, no. 6, pp. 1–49, 2021, IBM Research, "Watson AIOps: AI for IT operations," IBM, Technical White Paper, 2022.

[8] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," Proc. ICLR, 2023.

[9] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language Models Can Teach Themselves to Use Tools," Proc. NeurIPS, 2023.

[10] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent- Computer Interfaces Enable Automated Software Engineering," arXiv:2405.15793, 2024.

[11] Cognition AI, "Devin: The First AI Software Engineer," Technical Blog, Mar. 2024.

[12] K. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, Omega, and Kubernetes," ACM Queue, vol. 14, no. 1, pp. 70–93, Jan.–Feb. 2016.

[13] Y. Izrailevsky and A. Tseitlin, "The Netflix Simian Army," Netflix Technology Blog, 2011.

[14] U. Mukkara et al., "LitmusChaos: A chaos engineering framework for cloud-native workloads," in Proc. KubeCon +CloudNativeCon North America, 2020.

[15] T. Wilkie, "The RED Method: How to Instrument Your Services," presented at KubeCon + CloudNativeCon Europe, Copenhagen, Denmark, May 2018.

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:38:32 UTC from IEEE Xplore. Restrictions apply.